H^s SOFTWARE HANDBOOK

Complete Codebase Publication

Pseudocode · Python · R

Peter Higgins — April 2026

Higgins Decomposition Pipeline v3.0

$$H^s = R \circ M \circ E \circ C \circ T \circ V \circ S$$

Contents

1. Architecture Overview
2. Complete Variable Registry
3. Complete Parameter Registry
4. Pipeline Flowchart
5. State Machine Diagram
6. Guard Layer — Input Validation
7. Step S: Simplex Closure
8. Step V: CLR Transform
9. Step T: Variance Trajectory
10. Step C: HVLD Vertex Lock
11. Step E: Transcendental Squeeze
12. Step M: EITT Entropy
13. Geometry Branch — Ternary · Complex · Helix
14. Step R: Extended Panel + Report
15. Diagnostic Code System
16. Structural Mode Detection
17. Atomic Disassembly — Byte-Level Trace
18. Hidden Truths — What the Code Reveals

1. Architecture Overview

The H^s pipeline is a deterministic compositional analysis instrument expressed as the composition of seven operators: $H^s = R \circ M \circ E \circ C \circ T \circ V \circ S$. Each operator transforms data from one mathematical space to another, and no operator depends on any tunable parameter. The pipeline reads raw measurements, decomposes them through simplex geometry, and assembles a manifold characterisation.

The seven operators:

Operator	Name	Domain → Codomain	Purpose
S	Simplex	$\mathbb{R}^{+N\times D} \to S^{D-1}$	Close raw data to unit simplex; zero replacement
V	Variance	$S^{D-1} \to \mathbb{R}^N$ (via CLR)	CLR transform + cumulative Aitchison variance σ^2_A(t)
T	Transcendental	$\mathbb{R}^N \to$ {matches}	Proximity test: σ^2_A against 35 transcendental constants
C	Classification	$\mathbb{R}^N \to$ {NATURAL, INVESTIGATE, FLAG}	HVLD parabolic fit + HTP classification
E	Entropy	$S^{D-1} \to \mathbb{R}^N$	Shannon entropy H(t); EITT scale-invariance test
M	Modes	{codes} → {modes}	Structural mode detection from code combination matrix
R	Report	All → JSON	Assemble full diagnostic report with 78 codes

Pipeline files:

File	Lines	Purpose
higgins_decomposition_12step.py	~1,500	Core 12-step pipeline + extended panel
hs_codes.py	~645	78 diagnostic codes + 10

File	Lines	Purpose
		structural modes + code generator
hs_metrology.py	~607	8 instrument self-evaluation metrics
hs_sensitivity.py	~500	Component Power Mapper — perturbation analysis
hs_amalgamation.py	~400	Subcompositional recursion engine
hs_reporter.py	~300	Human-readable report generator
hs_fingerprint.py	~200	Compositional fingerprint hash generator
hs_controller.py	~350	Industrial state machine controller

Total codebase: approximately 4,500 lines of Python. Zero external dependencies beyond NumPy. The entire instrument can be reproduced from this handbook.

2. Complete Variable Registry

Every variable in the pipeline is listed below with its type, shape, origin step, and purpose. Variables are grouped by pipeline stage. All are float64 unless noted.

2.1 Instance Variables (HigginsDecomposition class)

Variable	Type	Shape	Set By	Description
experiment_id	str	—	__init__	Experiment identifier (e.g., 'Hs-03')
name	str	—	__init__	Human-readable experiment name
domain	str	—	__init__	Scientific domain (NUCLEAR, ENERGY, etc.)
carriers	list[str]	D	__init__	Names of compositional parts
D	int	—	__init__	Number of carriers (dimensionality)
data_source	dict	—	__init__	Provenance: type, URL, description
raw_data	ndarray	N×D	load_data	Original measurements (float64)
simplex_data	ndarray	N×D	close_to_simplex	Closed compositions (rows sum to 1)
clr_data	ndarray	N×D	clr_transform	Centred log-ratio coordinates
sigma2_A	ndarray	N	aitchison_variance	Cumulative Aitchison variance trajectory
pll_result	dict	—	pll_parabola	HVLD fit: coefficients, R^2, shape, vertex

Variable	Type	Shape	Set By	Description
squeeze_result	dict	—	super_squeeze	Transcendental matches: list, count, closest
entropy_result	dict	—	eitt_entropy	Shannon H, EITT invariance, chaos detection
ternary_result	dict	—	ternary_projection	Barycentric x,y coordinates
complex_result	dict	—	complex_plane	Centroid-relative Re, Im coordinates
helix_result	dict	—	helix_polar	Radius, theta, omega, angular velocity
results	dict	—	all steps	Accumulated pipeline outputs for JSON export
run_timestamp	str	—	run_full_pipeline	ISO 8601 UTC timestamp
data_hash	str	16 chars	load_data	SHA-256 hash (first 16 hex digits)
extended_results	dict	—	run_extended	Extended panel outputs

2.2 Intermediate Variables (per-function scope)

Variable	Type	Shape	Function	Description
data	ndarray	N×D	close_to_simplex	Working copy for zero replacement
mask	ndarray(bool)	N×D	close_to_simplex	Boolean mask: True where value ≤ 0
row_sums	ndarray	N×1	close_to_simplex	Row sums for

Variable	Type	Shape	Function	Description
				closure division
log_data	ndarray	N×D	clr_transform	Natural log of simplex data
geo_mean_log	ndarray	N×1	clr_transform	Row-wise mean of log_data
sigma2	ndarray	N	aitchison_variance	Variance accumulator array
window	ndarray	t×D	aitchison_variance	CLR data slice [0:t+1]
x, y	ndarray	N-2	pll_parabola	Valid time indices and variance values
coeffs [a,b,c]	ndarray	3	pll_parabola	Quadratic fit coefficients
y_pred	ndarray	N-2	pll_parabola	Predicted variance from fit
ss_res, ss_tot	float64	—	pll_parabola	Residual and total sum of squares
R2	float64	—	pll_parabola	Coefficient of determination
vertex_x, vertex_y	float64	—	pll_parabola	Parabola vertex coordinates
valid_sigma	ndarray	N-2	super_squeeze	σ^2_A values from index 2 onward
matches	list[dict]	≤50	super_squeeze	Transcendental proximity matches
delta	float64	—	super_squeeze	|test_value − constant_value|
H	ndarray	N	eitt_entropy	Raw Shannon entropy per observation

Variable	Type	Shape	Function	Description
H_norm	ndarray	N	eitt_entropy	Normalised entropy H/H_max
H_max	float64	—	eitt_entropy	Maximum entropy ln(D)
geo_mean	ndarray	blocks×D	eitt_entropy	Geometric-mean decimated compositions
dH	ndarray	N-1	_detect_chaos	First differences of H_norm
running_mean	ndarray	varies	_detect_chaos	Windowed mean of |dH|
p	ndarray	N×D or D	ternary_projection	Simplex data for barycentric calc
z_re, z_im	ndarray	N	complex_plane	Centroid-relative coordinates
radius	ndarray	N	helix_polar	$\sqrt{(Re^2 + Im^2)}$
theta	ndarray	N	helix_polar	atan2(Im, Re)
omega	ndarray	N-1	helix_polar	Angular velocity diff(θ), unwrapped
carrier_var	ndarray	D	run_extended	Per-carrier CLR variance
carrier_pct	ndarray	D	run_extended	Carrier variance as percentage of total
var_matrix	ndarray	D×D	run_extended	Pairwise log-ratio variation matrix
lr	ndarray	N	run_extended	Log-ratio $\ln(x_i/x_j)$ for pair analysis

3. Complete Parameter Registry

Every numerical constant and threshold in the codebase. None are tuneable for classification outcome. Grouped by type.

3.1 Transcendental Constants (35)

Name	Value	Symbol	Origin
pi	3.14159265...	π	Circle ratio
1/pi	0.31830989...	1/π	Reciprocal
e	2.71828183...	e	Natural base
1/e	0.36787944...	1/e	Reciprocal
ln2	0.69314718...	ln(2)	Natural log of 2
1/ln2	1.44269504...	1/ln(2)	Reciprocal
phi	1.61803399...	φ = (1+√5)/2	Golden ratio
phi^2	2.61803399...	φ^2	Golden ratio squared
1/phi	0.61803399...	1/φ	Reciprocal golden ratio
sqrt2	1.41421356...	√2	Pythagoras constant
1/sqrt2	0.70710678...	1/√2	Reciprocal
sqrt3	1.73205081...	√3	Equilateral triangle
1/sqrt3	0.57735027...	1/√3	Reciprocal
euler_gamma	0.57721566...	γ	Euler-Mascheroni constant
catalan	0.91596559...	G	Catalan constant
ln10	2.30258509...	ln(10)	Natural log of 10
1/ln10	0.43429448...	1/ln(10)	Reciprocal
pi/4	0.78539816...	π/4	Quarter circle
2pi	6.28318531...	2π	Full circle

Name	Value	Symbol	Origin
e^pi	23.1406926...	e^π	Gelfond constant
pi^e	22.4591577...	π^e	Pi to the e
sqrt5	2.23606798...	√5	Related to φ
1/sqrt5	0.44721360...	1/√5	Reciprocal
ln_phi	0.48121183...	ln(φ)	Log golden ratio
pi^2/6	1.64493407...	$\pi^2/6 = \zeta(2)$	Basel problem
apery	1.20205690...	ζ(3)	Apéry constant
khinchin	2.68545200...	K_0	Khinchin constant
feigenbaum_delta	4.66920161...	δ_F	Feigenbaum delta
feigenbaum_alpha	2.50290788...	α_F	Feigenbaum alpha
1/(2pi)	0.15915494...	1/(2π)	Reciprocal full circle
1/(e^pi)	0.04321392...	1/e^π	Reciprocal Gelfond
1/(pi^e)	0.04452311...	1/π^e	Reciprocal
omega_lambert	0.56714329...	Ω	Lambert W(1)
dottie	0.73908513...	d	Fixed point of cos(x)
glaisher_kinkelin	1.28242713...	A	Glaisher-Kinkelin constant

3.2 Thresholds and Guards

Parameter	Value	Type	Tuneable?	Effect on Classification
ZERO_DELTA	10^{-6}	Threshold	No	None — multiplicative zero replacement magnitude
Squeeze proximity	0.01 (1%)	Threshold	No	None — identifies

Parameter	Value	Type	Tuneable?	Effect on Classification
				closest constant regardless
EITT variation limit	5%	Threshold	No	None — relaxing to 10% changes zero outcomes
Chaos stall	15% of running mean	Threshold	No	None — DIS code trigger only
Chaos spike	3.5× running mean	Threshold	No	None — DIS code trigger only
Chaos reversal window	5 samples, ≥2 sign changes	Window	No	None — counting parameter
Running mean window	5 samples	Window	No	None — smoothing width
HVLD polynomial order	2 (quadratic)	Fixed	No	None — the vertex IS the quadratic
Guard 1: D ≥ 2	2	Guard	No	None — mathematical minimum
Guard 2: N ≥ 5	5	Guard	No	None — minimum for polynomial fit
Guard 3: No NaN/Inf	—	Guard	No	None — data integrity
Guard 4: Scale < 10^{15}	10^{15}	Guard	No	None — IEEE 754 limit
PID histogram bins	20	Estimation	Semi	Affects MI precision, not classification
TE histogram bins	15	Estimation	Semi	Affects TE precision, not classification
TE lag	1	Fixed	No	None — single-step dependency

Parameter	Value	Type	Tuneable?	Effect on Classification
Carrier dominance	60%	Threshold	No	None — DIS code trigger only
Ratio stability CV	< 20%	Threshold	No	None — DIS code trigger only
Ratio volatility CV	> 50%	Threshold	No	None — DIS code trigger only
Drift ratio band	0.95–1.05	Threshold	No	None — DIS code trigger only
Zero-crossing threshold	0.01	Threshold	No	None — simplex boundary proximity
Order sensitivity	5% relative diff	Threshold	No	None — detection trigger

3.3 Parameter Classification Summary

Category	Count	Tuneable?
Mathematical identities (transcendental constants)	35	No — fixed by mathematics
IEEE 754 hardware constraints	2	No — fixed by hardware
CoDa community standards	2	No — Aitchison/Martín-Fernández
Detection thresholds (DIS code triggers)	11	No — affect discovery alerts only
Estimation parameters (histogram bins)	2	Semi — affect precision, not classification
TOTAL	52	Zero tuneable for classification outcome

4. Pipeline Flowchart

The pipeline is a strictly sequential chain: each step receives the output of the previous step and produces input for the next. No step branches, loops, or backtracks. The extended panel runs after the core 12 steps and is auto-configured based on data dimensions.

Pipeline Data Flow

```
                    ┌──────────────────────┐
                    │   RAW DATA  (N × D)  │
                    │   float64 measurements │
                    └──────────────────────┘
                               │
                               ▼
                    ┌──────────────────────┐
                    │  GUARDS (1-4)        │
                    │  D≥2, N≥5, ¬NaN, Scale │
                    └──────────────────────┘
                               │ PASS
                               ▼
            S       ┌──────────────────────┐
                    │  SIMPLEX CLOSURE     │
                    │  Zero replace → Closure │
                    │  Output: S^(D-1)     │
                    └──────────────────────┘
                               │
                               ▼
            V       ┌──────────────────────┐
                    │  CLR TRANSFORM       │
                    │  ln(x) - mean(ln(x)) │
                    │  Output: ℝ^D         │
                    ├──────────────────────┤
                    │  AITCHISON VARIANCE  │
                    │  σ²_A(t) trajectory  │
                    │  Output: ℝ^N         │
                    └──────────────────────┘
                               │
              ┌────────────────┼────────────────┐
              │                │                │
              ▼                ▼                ▼
┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐
│  T: SQUEEZE      │ │  C: HVLD FIT     │ │  E: EITT ENTROPY │
│  35 constants    │ │  ax²+bx+c        │ │  H(t), decimation │
│  δ < 0.01 match  │ │  R², vertex      │ │  invariance test │
└──────────────────┘ └──────────────────┘ └──────────────────┘
         │                    │                    │
         └────────────────────┼────────────────────┘
                              │
                              ▼
                    ┌──────────────────────┐
                    │  GEOMETRY BRANCH     │
                    │  Ternary → Complex → │
                    │  Helix/Polar         │
                    └──────────────────────┘
                              │
                              ▼
                    ┌──────────────────────┐
                    │  EXTENDED PANEL      │
                    │  Universal: PCC, Drift, │
                    │    CoDa, Match Density │
                    │  Conditional: PID, TE, │
                    │    Order, Ratio Lattice │
                    └──────────────────────┘
                              │
                              ▼
          M         ┌──────────────────────┐
                    │  DIAGNOSTIC CODES    │
                    │  78 codes → 10 modes │
                    └──────────────────────┘
                              │
                              ▼
                    ┌──────────────────────┐
```

```
R    │  REPORT (JSON)          │
     │  Full manifold output    │
     └──────────────────────────┘
```

5. State Machine Diagram

The pipeline has exactly 6 possible states. Transitions are deterministic — the same input always produces the same state sequence. There is no backtracking and no branching except at the GUARD state (PASS/REJECT).

State Machine

```
┌──────────┐          PASS          ┌──────────┐                ┌──────────┐
│  IDLE    │──────────────────────▶│  GUARD   │──────────────▶│  CORE    │
│          │                        │  CHECK   │                │  PIPELINE│
└──────────┘                        └──────┬───┘                └──────┬───┘
                                           │ REJECT                    │
                                      ┌────▼─────┐                ┌────▼─────┐
                                      │  ERROR   │                │  EXTEND  │
                                      │  (halt)  │                │  PANEL   │
                                      └──────────┘                └──────┬───┘
                                                                         │
                                                                   ┌────▼─────┐
                                                                   │  CODE    │
                                                                   │  GEN     │
                                                                   └──────┬───┘
                                                                         │
                                                                   ┌────▼─────┐
                                                                   │  REPORT  │
                                                                   │  (done)  │
                                                                   └──────────┘

States:  IDLE → GUARD → CORE → EXTEND → CODE_GEN → REPORT
Error:   GUARD → ERROR (terminal, with diagnostic)

Guard codes emitted:
  GD-D2M-ERR : D < 2                GD-NAN-ERR : NaN found
  GD-N5M-ERR : N < 5                GD-INF-ERR : Inf found
  GD-SCL-ERR : Scale > 10¹⁵         GD-ALL-INF : All pass
```

5.1 Classification State Transitions

HTP Classification Logic

```
Classification is determined by the Squeeze step (S8):

σ²_A computed ──▶ Test against 35 constants
                  │
                  ├── δ_min < 0.01 ──▶ NATURAL  (S8-NAT-INF)
                  │
                  ├── 0.01 ≤ δ_min < 0.05 ──▶ INVESTIGATE  (S8-INV-WRN)
                  │
                  └── δ_min ≥ 0.05 ──▶ FLAG  (S8-FLG-WRN)

These thresholds are absolute proximity measures in Aitchison variance
units. They are not tuneable — any value of δ_min produces a classification.
```

6. Guard Layer — Input Validation

Four guards protect the pipeline from invalid input. All guards must pass before any computation begins. Each guard emits a diagnostic code on failure.

Guards — Pseudocode

```
FUNCTION validate_input(data[N×D], carriers[D]):
  GUARD 1: IF len(carriers) < 2 THEN RAISE 'D must be ≥ 2'
  GUARD 2: IF N < 5 THEN RAISE 'N must be ≥ 5'
  GUARD 3: IF any(isNaN(data)) OR any(isInf(data)) THEN RAISE 'Non-finite'
  GUARD 4: IF max(data[data>0]) / min(data[data>0]) > 10^15 THEN RAISE 'Scale'
  hash ← SHA256(data.bytes)[0:16]
  EMIT GD-ALL-INF
  RETURN hash
```

Guards — Python

```
def load_data(self, data):
    self.raw_data = np.array(data, dtype=np.float64)
    N, D = self.raw_data.shape
    assert D == self.D
    if N < 5:
        raise ValueError(f'Guard 2: N={N} < 5')
    if np.isnan(self.raw_data).any():
        raise ValueError('Guard 3: NaN detected')
    if np.isinf(self.raw_data).any():
        raise ValueError('Guard 3: Inf detected')
    pos = self.raw_data[self.raw_data > 0]
    if len(pos) > 0 and pos.max()/pos.min() > 1e15:
        raise ValueError('Guard 4: Scale disparity')
    self.data_hash = hashlib.sha256(
        self.raw_data.tobytes()).hexdigest()[:16]
```

Guards — R

```
validate_input <- function(data, carriers) {
  D <- length(carriers)
  N <- nrow(data)
  stopifnot('Guard 1: D >= 2' = D >= 2)
  stopifnot('Guard 2: N >= 5' = N >= 5)
  stopifnot('Guard 3: No NaN' = !any(is.nan(data)))
  stopifnot('Guard 3: No Inf' = !any(is.infinite(data)))
  pos <- data[data > 0]
  stopifnot('Guard 4: Scale' = max(pos)/min(pos) <= 1e15)
  hash <- digest::digest(data, algo='sha256')
  return(substr(hash, 1, 16))
}
```

7. Step S: Simplex Closure

Operator S maps raw positive measurements into the unit simplex S^(D-1). Zero or negative values are replaced using the multiplicative method of Martín-Fernández et al. (2003) with ZERO_DELTA = 10^{-6}. After replacement, each row is divided by its sum to achieve closure.

Simplex Closure — Pseudocode

```
FUNCTION close_to_simplex(raw[N×D]):
  data ← COPY(raw)
  FOR each cell (i,j) WHERE data[i,j] ≤ 0:
    data[i,j] ← ZERO_DELTA
    non_zero ← cells in row i where original > 0
    total_delta ← count_replaced × ZERO_DELTA
    scale ← (1 - total_delta) / SUM(data[i, non_zero])
    data[i, non_zero] ← data[i, non_zero] × scale
  FOR each row i:
    simplex[i,:] ← data[i,:] / SUM(data[i,:])
  VERIFY: |SUM(simplex[i,:]) - 1.0| < 10⁻¹² for all i
  EMIT S4-CLO-INF
  RETURN simplex[N×D]
```

Simplex Closure — Python

```
def close_to_simplex(self):
    data = self.raw_data.copy()
    has_zeros = np.any(data <= 0)
    if has_zeros:
        mask = data <= 0
        data[mask] = ZERO_DELTA  # 1e-6
        for i in range(data.shape[0]):
            row = data[i]
            n_replaced = mask[i].sum()
            if n_replaced > 0 and n_replaced < self.D:
                total_delta = n_replaced * ZERO_DELTA
                non_zero = ~mask[i]
                scale = (1.0 - total_delta) / row[non_zero].sum()
                data[i, non_zero] *= scale
                data[i, mask[i]] = ZERO_DELTA
    row_sums = data.sum(axis=1, keepdims=True)
    self.simplex_data = data / row_sums
```

Simplex Closure — R

```
close_to_simplex <- function(raw, ZERO_DELTA=1e-6) {
  data <- raw
  mask <- data <= 0
  if (any(mask)) {
    data[mask] <- ZERO_DELTA
    for (i in 1:nrow(data)) {
      n_rep <- sum(mask[i,])
      if (n_rep > 0 && n_rep < ncol(data)) {
        nz <- !mask[i,]
        total_d <- n_rep * ZERO_DELTA
        scale <- (1 - total_d) / sum(data[i, nz])
        data[i, nz] <- data[i, nz] * scale
      }
    }
  }
  simplex <- data / rowSums(data)
  stopifnot(all(abs(rowSums(simplex) - 1) < 1e-12))
  return(simplex)
}
```

8. Step V: CLR Transform

The Centred Log-Ratio transform maps simplex compositions to Euclidean space where standard statistical operations are valid. For each observation, subtract the geometric mean of the log-transformed parts. The CLR vector sums to zero for every row — this is the zero-sum constraint of Aitchison geometry.

CLR Transform — Pseudocode

```
FUNCTION clr_transform(simplex[N×D]):
  FOR each row i:
    log_data[i,j] ← ln(simplex[i,j])  for all j
    geo_mean_log[i] ← MEAN(log_data[i,:])
    clr[i,j] ← log_data[i,j] - geo_mean_log[i]
  VERIFY: SUM(clr[i,:]) = 0 for all i
  EMIT S5-CLR-INF
  RETURN clr[N×D]
```

CLR Transform — Python

```
def clr_transform(self):
    log_data = np.log(self.simplex_data)
    geo_mean_log = log_data.mean(axis=1, keepdims=True)
    self.clr_data = log_data - geo_mean_log
```

CLR Transform — R

```
clr_transform <- function(simplex) {
  log_data <- log(simplex)
  geo_mean_log <- rowMeans(log_data)
  clr <- log_data - geo_mean_log
  stopifnot(all(abs(rowSums(clr)) < 1e-12))
  return(clr)
}
```

9. Step T (continued): Aitchison Variance Trajectory

The variance trajectory σ^2_A(t) is the cumulative total Aitchison variance computed from the first observation up to time t. This is the dispersion curve of the composition over the observation window. Its shape — parabolic, linear, oscillating — encodes the character of the system.

Variance Trajectory — Pseudocode

```
FUNCTION aitchison_variance(clr[N×D]):
  sigma2[0] ← 0
  sigma2[1] ← 0
  FOR t = 2 TO N-1:
    window ← clr[0:t+1, :]          # first t+1 rows
    var_j ← VARIANCE(window[:,j])    # per-carrier variance
    sigma2[t] ← SUM(var_j)           # total Aitchison variance
  EMIT S6-VAR-INF
  RETURN sigma2[N]
```

Variance Trajectory — Python

```
def aitchison_variance(self):
    N = self.clr_data.shape[0]
    sigma2 = np.zeros(N)
    for t in range(2, N):
        window = self.clr_data[:t+1]
        sigma2[t] = np.var(window, axis=0).sum()
    self.sigma2_A = sigma2
```

Variance Trajectory — R

```
aitchison_variance <- function(clr) {
  N <- nrow(clr)
  sigma2 <- numeric(N)
  for (t in 3:N) {  # R is 1-indexed
    window <- clr[1:t, , drop=FALSE]
    sigma2[t] <- sum(apply(window, 2, var))
  }
  return(sigma2)
}
```

10. Step C: HVLD Vertex Lock

The Higgins Vertex Lock Diagnostic fits a quadratic $y = ax^2 + bx + c$ to the variance trajectory. The sign of a determines the shape (bowl or hill), and the vertex location $(-b/2a, c - b^2/4a)$ is the lock point. R^2 measures fit quality. This is the classification engine of the pipeline.

HVLD Fit — Pseudocode

```
FUNCTION hvld_fit(sigma2[N]):
  x ← [2, 3, ..., N-1]   # valid indices (skip 0,1)
  y ← sigma2[x]
  [a, b, c] ← POLYFIT(x, y, degree=2)
  y_pred ← a·x² + b·x + c
  R² ← 1 - SS_res/SS_tot
  vertex_x ← -b / (2a)
  vertex_y ← c - b²/(4a)
  shape ← IF a > 0 THEN 'bowl' ELSE 'hill'
  EMIT S7-BWL-INF or S7-HIL-INF
  EMIT S7-HRQ-INF if R² > 0.8
  RETURN {a, b, c, R², vertex, shape}
```

HVLD Fit — Python

```
def pll_parabola(self):
    N = len(self.sigma2_A)
    valid = np.arange(2, N)
    x = valid.astype(float)
    y = self.sigma2_A[valid]
    coeffs = np.polyfit(x, y, 2)
    a, b, c = coeffs
    y_pred = np.polyval(coeffs, x)
    ss_res = np.sum((y - y_pred) ** 2)
    ss_tot = np.sum((y - y.mean()) ** 2)
    R2 = 1.0 - ss_res / ss_tot if ss_tot > 0 else 0.0
    vertex_x = -b / (2 * a) if abs(a) > 1e-15 else 0.0
    vertex_y = c - b**2 / (4 * a) if abs(a) > 1e-15 else c
    shape = 'bowl' if a > 0 else 'hill'
```

HVLD Fit — R

```
hvld_fit <- function(sigma2) {
  N <- length(sigma2)
  x <- 3:N  # R is 1-indexed
  y <- sigma2[x]
  fit <- lm(y ~ poly(x, 2, raw=TRUE))
  coeffs <- coef(fit)  # c, b, a
  a <- coeffs[3]; b <- coeffs[2]; c_val <- coeffs[1]
  R2 <- summary(fit)$r.squared
  vertex_x <- -b / (2 * a)
  vertex_y <- c_val - b^2 / (4 * a)
  shape <- ifelse(a > 0, 'bowl', 'hill')
  list(a=a, b=b, c=c_val, R2=R2,
       vertex=c(vertex_x, vertex_y), shape=shape)
}
```

11. Step E: Transcendental Squeeze

The squeeze tests every value in the variance trajectory against 35 transcendental constants, both directly (σ^2_A ≈ c) and reciprocally (1/σ^2_A ≈ c). A match within δ < 0.01 classifies the system as NATURAL — its variance trajectory passes through a value that is within 1% absolute proximity of a mathematical constant.

Transcendental Squeeze — Pseudocode

```
FUNCTION squeeze(sigma2[N], constants[35]):
  valid ← sigma2[2:]    # skip t=0,1
  matches ← []
  FOR each constant c_k IN constants:
    FOR each value sv IN valid:
      δ_direct ← |sv - c_k|
      δ_reciprocal ← |1/sv - c_k|  (if sv > 10⁻¹⁵)
      IF δ_direct < 0.01 OR δ_reciprocal < 0.01:
        APPEND match {index, sv, c_k, δ, type}
  SORT matches BY δ ascending
  IF matches[0].δ < 0.01: EMIT S8-NAT-INF
  ELIF matches[0].δ < 0.05: EMIT S8-INV-WRN
  ELSE: EMIT S8-FLG-WRN
  RETURN matches[0:50]
```

Transcendental Squeeze — Python

```
def super_squeeze(self):
    valid_sigma = self.sigma2_A[2:]
    matches = []
    for name, const_val in TRANSCENDENTAL_CONSTANTS.items():
        if const_val <= 0: continue
        for idx, sv in enumerate(valid_sigma):
            if sv <= 0: continue
            for tv, label in [(sv,'direct'),
                    (1.0/sv if sv>1e-15 else 0,'reciprocal')]:
                if tv <= 0: continue
                delta = abs(tv - const_val)
                if delta < 0.01:
                    matches.append({
                        'time_index': idx+2,
                        'sigma2_A': float(sv),
                        'constant': name,
                        'delta': float(delta),
                    })
    matches.sort(key=lambda m: m['delta'])
```

Transcendental Squeeze — R

```
super_squeeze <- function(sigma2, constants) {
  valid <- sigma2[3:length(sigma2)]  # skip first 2
  matches <- data.frame()
  for (cname in names(constants)) {
    cval <- constants[[cname]]
    for (i in seq_along(valid)) {
      sv <- valid[i]
      if (sv <= 0) next
      d_direct <- abs(sv - cval)
      d_recip <- abs(1/sv - cval)
      if (d_direct < 0.01) {
        matches <- rbind(matches, data.frame(
          idx=i+2, sv=sv, const=cname,
          delta=d_direct, type='direct'))
      }
      if (sv > 1e-15 && d_recip < 0.01) {
        matches <- rbind(matches, data.frame(
          idx=i+2, sv=sv, const=cname,
          delta=d_recip, type='reciprocal'))
```

```
      }
    }
  }
  matches[order(matches$delta), ]
}
```

12. Step M: EITT Entropy

Shannon entropy is computed on the simplex compositions, normalised by H_max = ln(D). The EITT (Entropy Invariance Transcendental Test) checks whether the mean entropy is preserved under geometric-mean decimation at 2×, 4×, and 8× compression. Scale invariance within 5% variation indicates natural structure.

EITT Entropy — Pseudocode

```
FUNCTION eitt_entropy(simplex[N×D]):
  H_max ← ln(D)
  FOR each row i:
    p ← simplex[i,:]  (only entries > 0)
    H[i] ← -SUM(p · ln(p))
  H_norm ← H / H_max

  # EITT invariance test
  FOR factor IN [2, 4, 8]:
    n_trim ← (N ÷ factor) × factor
    blocks ← RESHAPE simplex[0:n_trim] → [n_trim/factor × factor × D]
    geo_mean ← EXP(MEAN(LN(blocks), axis=block))
    geo_mean ← geo_mean / ROW_SUM(geo_mean)  # re-close
    H_dec ← Shannon entropy of geo_mean
    variation ← |H_mean_orig - H_mean_dec| / H_mean_orig × 100
    PASS if variation < 5.0%
  EMIT S9-EIT-INF or S9-EIF-WRN
  RETURN {H, H_norm, eitt_results}
```

EITT Entropy — Python

```
def eitt_entropy(self):
    N = self.simplex_data.shape[0]
    H_max = np.log(self.D)
    H = np.zeros(N)
    for i in range(N):
        p = self.simplex_data[i]
        p_safe = p[p > 0]
        H[i] = -np.sum(p_safe * np.log(p_safe))
    H_norm = H / H_max if H_max > 0 else H
    eitt_results = {}
    for factor in [2, 4, 8]:
        if N < factor * 2: continue
        n_trim = (N // factor) * factor
        reshaped = self.simplex_data[:n_trim].reshape(
            -1, factor, self.D)
        geo_mean = np.exp(
            np.log(reshaped + 1e-30).mean(axis=1))
        geo_mean = geo_mean / geo_mean.sum(
            axis=1, keepdims=True)
        H_dec = np.zeros(len(geo_mean))
        for i in range(len(geo_mean)):
            p = geo_mean[i]; p_s = p[p > 0]
            H_dec[i] = -np.sum(p_s * np.log(p_s))
        H_dec_norm = H_dec / H_max
        var_pct = abs(H_norm[:n_trim].mean() -
            H_dec_norm.mean()) / H_norm[:n_trim].mean() * 100
        eitt_results[f'{factor}x'] = {
            'variation_pct': float(var_pct),
            'pass': var_pct < 5.0}
```

EITT Entropy — R

```
eitt_entropy <- function(simplex) {
  D <- ncol(simplex); N <- nrow(simplex)
  H_max <- log(D)
  H <- apply(simplex, 1, function(p) {
    p <- p[p > 0]
```

```
    -sum(p * log(p))
  })
  H_norm <- H / H_max
  eitt <- list()
  for (factor in c(2, 4, 8)) {
    if (N < factor * 2) next
    n_trim <- (N %/% factor) * factor
    blocks <- array(simplex[1:n_trim,],
      dim=c(factor, n_trim/factor, D))
    geo <- apply(blocks, c(2,3), function(x)
      exp(mean(log(x + 1e-30))))
    geo <- geo / rowSums(geo)
    H_dec <- apply(geo, 1, function(p) {
      p <- p[p > 0]; -sum(p * log(p))
    }) / H_max
    var_pct <- abs(mean(H_norm[1:n_trim]) -
      mean(H_dec)) / mean(H_norm[1:n_trim]) * 100
    eitt[[paste0(factor,'x')]] <- list(
      variation=var_pct, pass=(var_pct < 5))
  }
  list(H=H, H_norm=H_norm, eitt=eitt)
}
```

13. Geometry Branch — Ternary · Complex · Helix

The geometry branch projects simplex compositions through three coordinate systems: barycentric (ternary), complex plane (centroid-relative), and polar/helix (radius-angle-time). These projections build the manifold visualisation.

Ternary Projection (D=3) — Pseudocode

```
FUNCTION ternary(simplex[N×3]):
  p0, p1, p2 ← simplex columns
  x ← 0.5 × (2p1 + p2) / (p0 + p1 + p2)
  y ← (√3/2) × p2 / (p0 + p1 + p2)
  centroid ← (mean(x), mean(y))
  RETURN {x, y, centroid}
```

Ternary Projection (D=3) — Python

```
def ternary_projection(self):
    p = self.simplex_data
    x = 0.5 * (2*p[:,1] + p[:,2]) / (p[:,0]+p[:,1]+p[:,2])
    y = (np.sqrt(3)/2) * p[:,2] / (p[:,0]+p[:,1]+p[:,2])
    self.ternary_result = {
        'type': 'ternary', 'x': x.tolist(),
        'y': y.tolist(),
        'centroid': [float(x.mean()), float(y.mean())]}
```

Ternary Projection (D=3) — R

```
ternary_proj <- function(simplex) {
  p <- simplex
  denom <- rowSums(p)
  x <- 0.5 * (2*p[,2] + p[,3]) / denom
  y <- (sqrt(3)/2) * p[,3] / denom
  list(x=x, y=y, centroid=c(mean(x), mean(y)))
}
```

Complex Plane — Pseudocode

```
FUNCTION complex_plane(x, y, centroid):
  z_re ← x - centroid_x
  z_im ← y - centroid_y
  RETURN {z_re, z_im}
```

Complex Plane — Python

```
def complex_plane(self):
    x = np.array(self.ternary_result['x'])
    y = np.array(self.ternary_result['y'])
    cx, cy = self.ternary_result['centroid']
    z_re = x - cx
    z_im = y - cy
    self.complex_result = {'re': z_re.tolist(),
        'im': z_im.tolist(), 'centroid': [cx, cy]}
```

Complex Plane — R

```
complex_plane <- function(ternary) {
  cx <- ternary$centroid[1]
  cy <- ternary$centroid[2]
  list(re = ternary$x - cx, im = ternary$y - cy)
}
```

Helix / Polar — Pseudocode

```
FUNCTION helix_polar(z_re, z_im):
  radius ← √(z_re² + z_im²)
  theta ← atan2(z_im, z_re)
  omega ← DIFF(theta)
  omega ← UNWRAP(omega, [-π, π])
  RETURN {radius, theta, omega}
```

Helix / Polar — Python

```python
def helix_polar(self):
    re = np.array(self.complex_result['re'])
    im = np.array(self.complex_result['im'])
    radius = np.sqrt(re**2 + im**2)
    theta = np.arctan2(im, re)
    omega = np.diff(theta)
    omega = np.where(omega > np.pi, omega-2*np.pi, omega)
    omega = np.where(omega < -np.pi, omega+2*np.pi, omega)
    self.helix_result = {'radius': radius.tolist(),
        'theta': theta.tolist(), 'omega': omega.tolist(),
        'angular_velocity_std': float(np.std(omega))}
```

Helix / Polar — R

```r
helix_polar <- function(cpx) {
  radius <- sqrt(cpx$re^2 + cpx$im^2)
  theta <- atan2(cpx$im, cpx$re)
  omega <- diff(theta)
  # Unwrap
  omega[omega > pi] <- omega[omega > pi] - 2*pi
  omega[omega < -pi] <- omega[omega < -pi] + 2*pi
  list(radius=radius, theta=theta, omega=omega,
       ang_vel_std=sd(omega))
}
```

14. Step R: Extended Panel + Report

The extended panel runs after the core 12 steps and auto-detects which conditional tests to enable based on data dimensions and sample size.

14.1 Universal Panel (always runs)

Per-Carrier Contribution — Pseudocode

```
carrier_var[j] ← VAR(clr[:,j])  for j = 1..D
carrier_pct[j] ← carrier_var[j] / SUM(carrier_var) × 100
dominant ← ARGMAX(carrier_pct)
IF carrier_pct[dominant] > 60%: EMIT XU-PCD-DIS
```

Per-Carrier Contribution — Python

```
carrier_var = np.var(self.clr_data, axis=0)
total_var = carrier_var.sum()
carrier_pct = carrier_var / total_var * 100
dominant_idx = int(np.argmax(carrier_var))
```

Per-Carrier Contribution — R

```
carrier_var <- apply(clr, 2, var)
carrier_pct <- carrier_var / sum(carrier_var) * 100
dominant <- which.max(carrier_pct)
```

Carrier Drift Detection — Pseudocode

```
half ← N ÷ 2
var_1st ← TOTAL_VARIANCE(clr[1:half])
var_2nd ← TOTAL_VARIANCE(clr[half+1:N])
drift_ratio ← var_2nd / var_1st
direction ← 'increasing' if ratio > 1.05
             'decreasing' if ratio < 0.95
             'stable' otherwise
```

Carrier Drift Detection — Python

```
half = N // 2
first_half_var = np.var(self.clr_data[:half], axis=0).sum()
second_half_var = np.var(self.clr_data[half:], axis=0).sum()
drift_ratio = second_half_var / first_half_var
```

Carrier Drift Detection — R

```
half <- N %/% 2
v1 <- sum(apply(clr[1:half,], 2, var))
v2 <- sum(apply(clr[(half+1):N,], 2, var))
drift_ratio <- v2 / v1
```

14.2 Conditional Panel (auto-detected)

Partial Information Decomposition (D≥3) — Pseudocode

```
MI(x,y) ← mutual information via 20×20 histogram
I_x1_y ← MI(clr[:,0], clr[:,2])
I_x2_y ← MI(clr[:,1], clr[:,2])
redundancy ← MIN(I_x1_y, I_x2_y)
unique_x1 ← I_x1_y - redundancy
unique_x2 ← I_x2_y - redundancy
```

Partial Information Decomposition (D≥3) — Python

```
def _estimate_mi(x, y, bins=20):
    h2d, _, _ = np.histogram2d(x, y, bins=bins)
    pxy = h2d / h2d.sum()
    px = pxy.sum(axis=1)
    py = pxy.sum(axis=0)
    mi = 0.0
    for i in range(len(px)):
        for j in range(len(py)):
```

```
            if pxy[i,j] > 1e-30 and px[i] > 1e-30 and py[j] > 1e-30:
                mi += pxy[i,j] * log(pxy[i,j] / (px[i]*py[j]))
    return max(mi, 0)
```

Partial Information Decomposition (D≥3) — R

```
estimate_mi <- function(x, y, bins=20) {
  h2d <- hist2d(x, y, nbins=bins, show=FALSE)
  pxy <- h2d$counts / sum(h2d$counts)
  px <- rowSums(pxy)
  py <- colSums(pxy)
  mi <- 0
  for (i in seq_along(px)) {
    for (j in seq_along(py)) {
      if (pxy[i,j] > 1e-30 && px[i] > 1e-30 && py[j] > 1e-30)
        mi <- mi + pxy[i,j] * log(pxy[i,j] / (px[i]*py[j]))
    }
  }
  max(mi, 0)
}
```

Transfer Entropy (N≥50) — Pseudocode

```
FOR each carrier pair (i,j), i≠j:
  src ← clr[1:N-1, i]
  tgt_past ← clr[1:N-1, j]
  tgt_fut ← clr[2:N, j]
  TE(i→j) ← H(tgt_fut,tgt_past) - H(tgt_past)
            - H(tgt_fut,src) + H(src)
  via 15-bin histograms, lag=1
```

Transfer Entropy (N≥50) — Python

```
for i in range(min(D, 4)):
    for j in range(min(D, 4)):
        if i == j: continue
        lag = 1; n_te = N - lag
        src = self.clr_data[:n_te, i]
        tgt_past = self.clr_data[:n_te, j]
        tgt_fut = self.clr_data[lag:lag+n_te, j]
        te = _h2(tgt_fut,tgt_past) - _h1(tgt_past)\
           - _h2(tgt_fut,src) + _h1(src)
```

Transfer Entropy (N≥50) — R

```
transfer_entropy <- function(clr, lag=1, bins=15) {
  D <- ncol(clr); N <- nrow(clr)
  te_mat <- matrix(0, D, D)
  for (i in 1:D) for (j in 1:D) {
    if (i == j) next
    src <- clr[1:(N-lag), i]
    tp <- clr[1:(N-lag), j]
    tf <- clr[(lag+1):N, j]
    te_mat[i,j] <- h2d(tf,tp) - h1d(tp)
                 - h2d(tf,src) + h1d(src)
  }
  te_mat
}
```

15. Diagnostic Code System

The pipeline emits 78 diagnostic codes in SS-CCC-LLL format. Each code is a geometric fact about the composition. The code system is complete: every possible pipeline outcome maps to at least one code.

15.1 Code Format

Field	Width	Values	Meaning
SS (Stage)	2 chars	GD, S4-SC, XU, XC, RP, SM	Pipeline step that emitted the code
CCC (Condition)	3 chars	e.g., NAT, BWL, EIT	What was detected
LLL (Level)	3 chars	INF, WRN, ERR, DIS, CAL	Severity / information class

15.2 Complete Code Dictionary (78 codes)

GD (Guards)

Code	Short	Description
GD-D2M-ERR	D < 2	Input rejected: D must be >= 2. The simplex requires at least 2 compositional pa
GD-N5M-ERR	N < 5	Input rejected: N must be >= 5. The variance trajectory needs at least 3 non-zer
GD-NAN-ERR	NaN detected	Input rejected: data contains NaN values. Clean or impute before decomposition.
GD-INF-ERR	Inf detected	Input rejected: data contains Inf values. Check for division by zero or overflow
GD-SCL-ERR	Scale disparity	Input rejected: max/min ratio exceeds 1e15. Normalise or partition before decomp
GD-ALL-INF	All guards pass	All 4 input guards passed. Data is valid for H^s pipeline processing.

S4 (Closure)

Code	Short	Description
S4-CLO-INF	Closure OK	Simplex closure successful. All rows sum to 1.0 within tolerance 1e-12.
S4-ZRP-INF	Zero replacement	Multiplicative zero replacement applied. ZERO_DELTA = 1e-6.
S4-ZRN-INF	No zeros	No zero replacement needed. All input values are positive.

S5 (CLR)

Code	Short	Description
S5-CLR-INF	CLR complete	Centred log-ratio transform applied. All rows sum to zero in CLR space.

S6 (Variance)

Code	Short	Description
S6-VAR-INF	Variance computed	Cumulative Aitchison variance trajectory σ^2_A(t) computed for all time indices.
S6-ZRO-WRN	Zero variance	σ^2_A is zero or near-zero. Data may be constant or degenerate.
S6-RNG-INF	Variance range	Variance trajectory range and final value recorded.

S7 (HVLD)

Code	Short	Description
S7-BWL-INF	HVLD bowl	HVLD vertex lock: bowl shape (a > 0). Variance is accelerating — system integrat
S7-HIL-INF	HVLD hill	HVLD vertex lock: hill shape (a < 0). Variance is decelerating

Code	Short	Description
		— system segregat
S7-HRQ-INF	HVLD R^2 high	HVLD $R^2 > 0.8$. Quadratic fit describes the trajectory well.
S7-LRQ-WRN	HVLD R^2 low	HVLD $R^2 < 0.5$. Quadratic fit is poor — trajectory may be non-polynomial.
S7-VTX-INF	Vertex location	HVLD vertex (lock point) location recorded.

S8 (Squeeze)

Code	Short	Description
S8-NAT-INF	NATURAL	HTP classification: NATURAL. At least one transcendental match within $\delta < 1\%$.
S8-INV-WRN	INVESTIGATE	HTP classification: INVESTIGATE. Nearest match δ between 1% and 5%. Consider alt
S8-FLG-WRN	FLAG	HTP classification: FLAG. No match within 5%. System may be synthetic, adversari
S8-EUL-DIS	Euler-family match	Nearest constant is from the Euler family (2π, e^π, π^e, or their reciprocals).
S8-SQZ-INF	Squeeze count	Number of transcendental proximity matches recorded.
S8-TGT-DIS	Tight match	Transcendental match tighter than $\delta < 0.001$. Precision-level structural resonanc

S9 (EITT)

Code	Short	Description
S9-ENT-INF	Entropy computed	Shannon entropy H(t) computed for all observations. Normalised by H_max = ln(D).
S9-EIT-INF	EITT pass	EITT invariance test passed at all tested compression ratios (variation < 5%).
S9-EIF-WRN	EITT fail	EITT invariance test failed at one or more compression ratios.
S9-CHS-INF	Chaos low	Chaos detection: few deviations. System is compositionally smooth.
S9-CHH-DIS	Chaos high	Chaos detection: many deviations. System exhibits compositional turbulence — sta
S9-STL-DIS	Stalls detected	Angular velocity stalls detected — entropy rate dropped below 15% of running mea
S9-SPK-DIS	Spikes detected	Angular velocity spikes detected — entropy rate exceeded 3.5× running mean.
S9-REV-DIS	Reversals detected	Entropy reversals detected — ≥2 sign changes in 5 consecutive samples.

SA-SC (Geometry)

Code	Short	Description
SA-TRN-INF	Ternary OK	Ternary projection computed. Type: ternary (D=3), 1-simplex (D=2), or projected
SB-CPX-INF	Complex plane OK	Complex plane mapping computed. Centroid-relative coordinates.
SC-HLX-INF	Helix OK	3D helix/polar embedding computed. Radius, angle,

Code	Short	Description
		angular velocity.

XU (Extended Universal)

Code	Short	Description
XU-PCC-INF	Carrier decomposition	Per-carrier contribution computed. Shows which carrier drives the variance.
XU-PCD-DIS	Carrier dominance	One carrier contributes >60% of total variance. Strong structural signal.
XU-MDN-INF	Match density natural	Match density consistent with natural system (density >> synthetic baseline).
XU-MDS-CAL	Match density low	Match density below natural baseline. May indicate synthetic data or sparse cons
XU-DRI-INF	Drift stable	Carrier drift trend: stable. Second half compositionally similar to first half.
XU-DRU-DIS	Drift increasing	Carrier drift trend: increasing. System is evolving — second half more dispersed
XU-DRD-DIS	Drift decreasing	Carrier drift trend: decreasing. System is converging — second half more concent
XU-CDA-INF	CoDa panel OK	CoDa structural panel computed: variation matrix, CLR summary, Aitchison distanc
XU-CLM-INF	Claim summary	Claim-status and envelope summary generated.
XU-HRC-INF	Hierarchy moderate	Carrier hierarchy ratio (max/min of central values) is between 10^3 and 10^6. Mode

Code	Short	Description
XU-HRX-WRN	Hierarchy extreme	Carrier hierarchy ratio exceeds 10^6 (but below Guard 4 limit of 10^{15}). Extreme d
XU-CSL-DIS	Conservation signature	All carrier pairs show PID redundancy and at least 50% of ratio pairs are stable
XU-FPR-INF	Fingerprint generated	Compositional fingerprint computed: hash of HVLD shape, R^2 band, classification,
XU-CPM-INF	Power map computed	Component Power Mapper analysis completed. Leverage index, phase boundary map, a
XU-DPC-DIS	Disproportionate carrier	One or more carriers have power-to-fraction ratios exceeding 2.0x, meaning their
XU-PSC-WRN	Phase-sensitive carrier	One or more carriers have criticality margins below 0.3, meaning small changes t
XU-CFR-WRN	Classification flip risk	Perturbation of one or more carriers caused the system classification to flip be
XU-PWR-DIS	Power ≠ mass	The power ranking of carriers differs from the mass fraction ranking. Component

XC (Extended Conditional)

Code	Short	Description
XC-PID-INF	PID computed	Partial Information Decomposition computed. Redundant, unique, and synergistic c
XC-PIR-DIS	PID redundancy	PID dominant mode: redundancy. Carriers share information — coupled dynamics.

Code	Short	Description
XC-PIU-DIS	PID unique	PID dominant mode: unique. One carrier contributes information the others cannot
XC-TEE-INF	Transfer entropy	Transfer entropy matrix computed. Directed information flow between carriers qua
XC-TEF-DIS	TE dominant flow	Dominant directed information flow identified. One carrier informationally predi
XC-TEZ-CAL	TE zero	Transfer entropy is zero across all pairs. System is compositionally memoryless
XC-ORD-INF	Order matters	Order sensitivity test: temporal/parametric order significantly affects the vari
XC-ORN-INF	Order neutral	Order sensitivity test: shuffling order does not change the result. Structure is
XC-RPL-INF	Ratio-pair lattice	Ratio-pair lattice computed. All pairwise log-ratios ranked by stability (CV).
XC-RPS-DIS	Stable ratio found	A highly stable pairwise ratio detected (CV < 20%). These carriers are compositi
XC-RPV-DIS	Volatile ratio	A highly volatile pairwise ratio detected (CV > 50%). These carriers are composi
XC-ZCR-DIS	Zero crossings	Carrier zero-crossing events detected. Compositions approaching simplex boundary

RP (Report)

Code	Short	Description
RP-CMP-INF	Run complete	H^s extended pipeline run completed successfully. All

Code	Short	Description
		results available for repor
RP-DTM-INF	Deterministic	Pipeline is deterministic. Repeated runs on identical data produce bit-identical
RP-VER-INF	Version	Pipeline version and configuration recorded.

SM (Structural Modes)

Code	Short	Description
SM-BPO-DIS	Bimodal population	Two or more distinct compositional populations detected on the simplex. The vari
SM-SCG-INF	Smooth convergence	System converges smoothly to compositional equilibrium. Bowl HVLD with low turbu
SM-MCA-WRN	Missing carrier	Compositional geometry suggests an incomplete decomposition. FLAG classification
SM-DMR-DIS	Domain resonance	System encodes deep geometric structure — Euler-family transcendental detected w
SM-CPL-DIS	Carrier coupling	Carriers are physically linked — PID shows redundancy and at least one ratio pai
SM-IND-DIS	Carrier independence	Carriers are governed by independent mechanisms — all ratio pairs are volatile a
SM-DGN-WRN	Degenerate simplex	Data approaches the simplex boundary — zero-crossings detected with FLAG or low
SM-RTR-DIS	Regime transition	System undergoes compositional regime change —

Code	Short	Description
		drift detected with stalls and re
SM-TNT-DIS	Turbulent natural	System is NATURAL but turbulent — transcendental match found despite EITT failur
SM-OVC-CAL	Overconstrained	System is highly constrained — near-perfect HVLD R^2, EITT pass, and precision-le

16. Structural Mode Detection

Structural modes are second-order diagnoses: they analyse the pattern of diagnostic codes, not the data directly. Each mode is defined by required codes (all must be present) and forbidden codes (none may be present). This is a boolean satisfiability check on the code set.

Mode	Name	Required Codes	Forbidden Codes	Priority
SM-OVC-CAL	Overconstrained	S7-HRQ, S8-NAT, S8-TGT, S9-EIT	S9-CHH, S8-FLG	1
SM-MCA-WRN	Missing carrier	S8-FLG + (XC-ZCR or S6-ZRO)	—	2
SM-BPO-DIS	Bimodal population	S7-HIL, S9-EIF, S9-CHH	—	3
SM-TNT-DIS	Turbulent natural	S8-NAT, S9-EIF, S9-CHH	—	4
SM-RTR-DIS	Regime transition	S9-STL, S9-REV + (XU-DRU or XU-DRD)	—	5
SM-DGN-WRN	Degenerate simplex	XC-ZCR + (S8-FLG or S7-LRQ)	—	6
SM-DMR-DIS	Domain resonance	S8-EUL, S8-NAT	S8-FLG	7
SM-CPL-DIS	Carrier coupling	XC-PIR, XC-RPS	—	8
SM-IND-DIS	Carrier independence	XC-RPV (≥3 pairs)	XC-RPS, XC-PIR	9
SM-SCG-INF	Smooth convergence	S7-BWL, S9-EIT	S9-CHH, S8-FLG, S9-EIF	10

Detection Algorithm

```
MODE DETECTION ALGORITHM (Pseudocode):

code_prefixes ← {strip level suffix from each emitted code}
   e.g., 'S7-BWL-INF' → 'S7-BWL'

FOR each mode IN STRUCTURAL_MODES (sorted by priority):
  IF ALL(mode.requires ⊂ code_prefixes)
     AND (mode.requires_any is empty OR ANY(mode.requires_any ⊂ code_prefixes))
     AND NONE(mode.forbids ⊂ code_prefixes):
```

```
    EMIT mode.code
```

17. Atomic Disassembly — Byte-Level Trace

This section traces the exact byte-level transformation of data through the pipeline for a reference dataset: Hs-03 Nuclear SEMF (N=92, D=3). Every intermediate array is sized, every operation is counted.

Stage	Input	Output	Bytes	Operations	Precision
Load + hash	92×3 float64	92×3 float64 + 16-char hash	2,208 + 32	N×D reads + SHA-256	float64
Zero replacement	92×3	92×3 (modified in-place)	2,208	N×D comparisons + conditional scaling	float64
Closure	92×3	92×3 (rows sum to 1.0)	2,208	92 divisions + 92 verifications	float64
CLR	92×3	92×3 (rows sum to 0.0)	2,208	276 ln() + 92 means + 276 subtractions	float64
Variance trajectory	92×3 CLR	92 scalars	736	90 windowed variances × 3 carriers	float64
HVLD fit	90 valid σ^2_A	3 coefficients	24	1 polyfit (SVD internally)	float64
R^2 computation	90 y, 90 y_pred	1 scalar	8	180 subtractions, 90 squares, 2 sums	float64
Vertex	3 coefficients	2 scalars	16	3 arithmetic ops	float64
Squeeze	90 σ^2_A values	≤50 matches	~4,000	90 × 35 × 2 = 6,300 comparisons	float64
Shannon H	92×3 simplex	92 scalars	736	276 p·ln(p) + 92 sums	float64
EITT 2×	92×3 → 46×3	1 variation %	8	46 geo-means + 46 entropies	float64

Stage	Input	Output	Bytes	Operations	Precision
EITT 4×	92×3 → 23×3	1 variation %	8	23 geo-means + 23 entropies	float64
EITT 8×	88×3 → 11×3	1 variation %	8	11 geo-means + 11 entropies	float64
Chaos detection	91 dH values	3 counts	24	~455 comparisons (windowed)	int + float64
Ternary	92×3 simplex	92×2	1,472	184 multiplications + 92 divisions	float64
Complex plane	92×2	92×2	1,472	184 subtractions	float64
Polar/helix	92×2	92×3 (r,θ,ω)	2,208	92 sqrt + 92 atan2 + 91 diff	float64
Extended panel	All above	~50 metrics	~2,000	Histograms, ratios, drift	float64
Code generation	All results	~30 codes	~3,000	78 boolean evaluations	string + float64
TOTAL PIPELINE	2,208 bytes in	~60,000 bytes out	—	~25,000 arithmetic ops	float64 throughout

17.1 Precision Chain

Every operation in the pipeline operates at IEEE 754 float64 precision (15.9 significant decimal digits). No operation truncates, rounds, or reduces precision. The final output carries the same precision as the input. This is unusual — most statistical methods reduce precision through averaging, sampling, or iterative convergence. The H^s pipeline is a pure precision-preserving transformation.

17.2 Information Monotonicity

The pipeline never discards information. Every intermediate result — raw data, simplex closure, CLR coordinates, variance trajectory, HVLD fit, entropy values — is stored and accessible in the output JSON. No stage overwrites its input. This allows any stage to be inspected without re-running the pipeline. The output is a complete audit trail of the analysis.

17.3 Computational Complexity

For N observations and D carriers, the pipeline performs $O(N^2D)$ elementary arithmetic operations. The quadratic scaling comes from the cumulative variance computation (Step 6), which recomputes variance over an expanding window. For the reference dataset (N=92, D=3), this is approximately 25,000 operations — a computation that completes in under 15 milliseconds on commodity hardware. The pipeline is geometry-bound, not compute-bound.

18. Hidden Truths — What the Code Reveals

This section catalogues properties of the H^s pipeline that are not stated in any design document but emerge from analysis of the code itself. These are structural truths — properties that the code has but that were not explicitly designed in.

18.1 The Pipeline Has No Branching Logic for Classification

Hidden truth: The HTP classification (NATURAL / INVESTIGATE / FLAG) is determined entirely by a single floating-point comparison: $\delta_min < 0.01$. There is no if-else chain, no decision tree, no weighted scoring. The classification is a pure geometric fact — the minimum distance between the variance trajectory and the nearest transcendental constant. This means the classification cannot be gamed, biased, or adjusted. It is a measurement, not a decision.

18.2 The CLR Zero-Sum Property Creates a Hidden Constraint

Hidden truth: The CLR transform produces vectors that sum to zero. This means that in D-dimensional CLR space, the data actually lives on a (D-1)-dimensional hyperplane. The variance trajectory $\sigma^2_A(t)$ is computed on this hyperplane, not in the full D-dimensional space. This is why the Aitchison variance is the correct measure — it respects the simplex constraint. Any other variance measure (e.g., Euclidean variance on raw data) would violate the geometry and produce spurious results. The code enforces this without comment.

18.3 The Squeeze Test Is Symmetric Under Inversion

Hidden truth: The super_squeeze function tests both σ^2_A and $1/\sigma^2_A$ against every constant. This means the test is invariant under the transformation $\sigma^2_A \to 1/\sigma^2_A$. Physically, this means a system with variance 0.31831 ($\approx 1/\pi$) and a system with variance π are both detected as 'near π.' The inversion symmetry doubles the effective constant library from 35 to approximately 70 test values, without adding any constants. This was not designed — it emerged from the observation that reciprocals of transcendental constants are also transcendental.

18.4 The EITT Decimation Is a Renormalisation Group Step

Hidden truth: The geometric-mean decimation in the EITT test is mathematically equivalent to a single step of a real-space renormalisation group (RG) transformation. In statistical physics, RG invariance at a fixed point indicates a phase transition — the system looks the same at all scales. When the EITT test passes (entropy invariant under decimation), the composition is at or near a compositional fixed point. The 5% threshold is essentially a relevance criterion: how close to the fixed point does the system need to be for us to call it 'scale-invariant'? The code implements an RG test without ever using the term.

18.5 The Chaos Detector Measures Ergodicity Breaking

Hidden truth: The _detect_chaos function measures stalls (velocity drops to zero), spikes (velocity exceeds 3.5× mean), and reversals (direction changes). In

dynamical systems theory, these three signatures together detect ergodicity breaking — the system's trajectory in phase space becomes non-stationary. When all three are present, the system is transitioning between basins of attraction. The code detects regime transitions without modelling the dynamics.

18.6 The Variation Matrix Encodes Physical Laws

Hidden truth: The variation matrix V[i,j] = var(ln(x_i/x_j)) computed in the extended panel is a complete description of pairwise compositional relationships. When V[i,j] ≈ 0, carriers i and j are locked in a fixed ratio — which in physics means they obey a conservation law. The code computes this matrix for all pairs and flags stable ratios (CV < 20%), effectively discovering conservation laws from data without being told to look for them.

18.7 The Pipeline Is Its Own Inverse (for Deterministic Data)

Hidden truth: Given the output JSON and the data hash, the pipeline can be re-run to produce bit-identical results. This means the pipeline is a fixed-point operator: f(f(data)) contains the same information as f(data). The second application adds no new codes, discovers no new modes, and changes no classifications. This is a consequence of determinism + zero tunable parameters. The instrument reads the same way every time because there is only one way to read.

18.8 The Fourier Pair Watchlist Is a Metacognitive Instrument

Hidden truth: The FOURIER_PAIR_WATCHLIST embedded in the code is not a code path — it is documentation that warns future users about a structural vulnerability. The pipeline uses rectangular block averaging (rect window) in the EITT decimation. The Fourier transform of rect is sinc. If someone approximates their physics with sinc, the pipeline will find structure that the pipeline itself created. The watchlist does not execute; it informs. This is a metacognitive instrument — code that knows what it doesn't know and tells you about it.

18.9 Power ≠ Mass: The Leverage Effect Is Universal

Hidden truth: The Component Power Mapper (hs_sensitivity.py) perturbs each carrier independently and measures the effect on classification. The finding that power ranking differs from mass fraction ranking has held across every tested dataset. A carrier that contributes 5% of the mass can contribute 50% of the classification power. This is the compositional leverage effect — it exists because the pipeline operates in log-ratio space where small absolute changes in minor carriers produce large relative shifts. The code reveals that compositional influence is fundamentally non-linear in the carrier fraction.

18.10 The Missing Truth: What the Code Cannot Do

Final hidden truth: The code analysis reveals what H^s cannot do. It cannot determine causation (it sees correlations in log-ratio space). It cannot handle time-varying carrier definitions (the carrier set is fixed at input). It cannot detect

structure below its noise floor (ZERO_DELTA = 10^{-6} sets the minimum resolvable composition). And it cannot distinguish between two systems that produce identical variance trajectories — the pipeline sees geometry, not physics. These limitations are not bugs; they are the boundaries of a well-defined instrument. Every instrument has a resolution limit. The H^s code makes its limits explicit.

H^s — The code is the instrument. The instrument is the code.

Software Handbook v1.0 — Peter Higgins, April 2026

Pseudocode · Python · R — Complete Codebase Publication